

Pratique de la Data Science

Cours 4 : Natural Language Processing (NLP)

Université Paris Dauphine – Master 2 Ingénierie Statistique et Financière (ISF)

Pierre Fihey – *pierre.fihey@ip-paris.fr*

Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

V. L'architecture Transformers

- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

V. L'architecture Transformers

- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

Introduction au NLP

Objectif : Créer des modèles capables de comprendre, manipuler et générer du langage naturel.

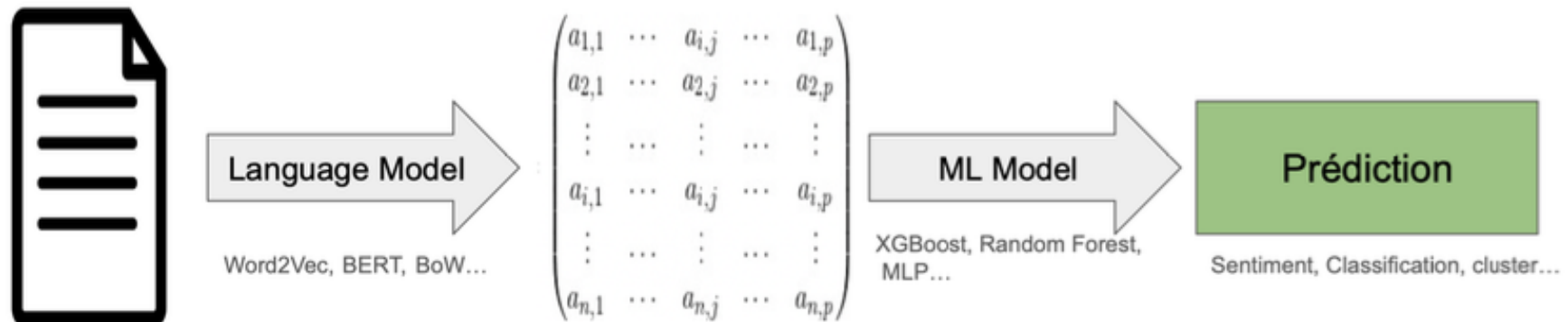
Applications :

- **Analyse de texte.**
 - Analyse de sentiments
 - Extraction d'informations, classification de documents...
 - Détection de contenu haineux, fake news...
- **Génération de texte.**
 - Systèmes conversationnels.
- **Transformation de texte.**
 - Génération de résumés.
 - Traduction de texte.
 - Transfert de style.

Introduction au NLP

Objectif d'un modèle de NLP :

- Représenter les données textuelles sous la forme de vecteurs capturant leur sémantique, leur structure syntaxique et leur contexte.



Exemple de pipeline pour une analyse textuelle

Principaux défis : Ambiguïté du langage, biais, longueur des séquences...

Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

V. L'architecture Transformers

- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

Bag of Words (BoW)

Principe : On représente chaque document par un vecteur où chaque dimension correspond à un mot du vocabulaire. On représente la fréquence de chaque mot dans le document.

I walked down the street
I walk down the the avenue
I walked down the avenue
I ran down the street
I walk down the city

	I	the	down	walked	street	avenue	walk	ran	city
D_1	1	1	1	1	1	0	0	0	0
D_2	1	2	1	0	0	1	1	0	0
D_3	1	1	1	1	0	1	0	0	0
D_4	1	1	1	0	1	0	0	1	0
D_5	1	1	1	0	0	0	1	0	1

Bag of Words (BoW)

Première représentation vectorielle de documents : Permet de mesurer des similarités entre documents.

Similarité cosinus :

- Mesure de similarité la plus utilisée en NLP. Non biaisée par la longueur des documents.

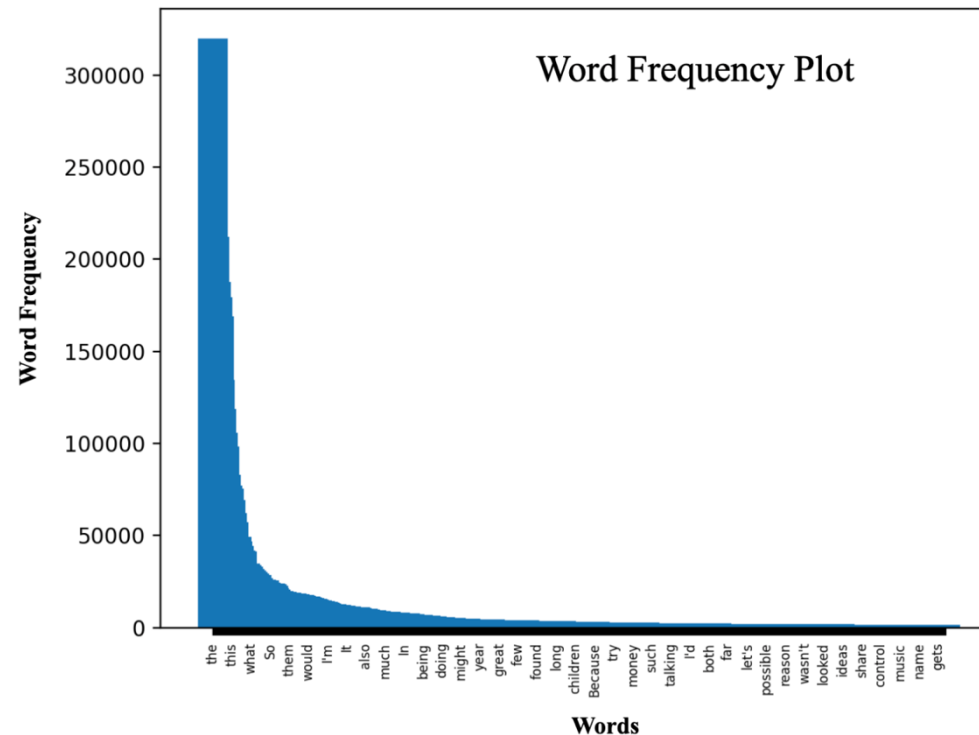
$$\text{sim}(D_1, D_2) = \frac{D_1 \cdot D_2}{\|D_1\| \times \|D_2\|}$$

Limites de BoW:

- Ne prend pas en compte l'ordre des mots, les relations sémantiques (mêmes distances entre tous les mots).
- Trop large vocabulaire $\Rightarrow$ "Fléau de la dimension".
- Les mots très fréquents sont rarement les plus importants.

Zipf's Law

Zipf's Law : Décrit la distribution des fréquences des mots dans un corpus de texte.



- Quelques mots sont très fréquents (stopwords...).
- La majorité des mots apparaissent rarement.
⇒ Fréquence de mots n'est pas la meilleure mesure ⇒ **Mots plus rares sont souvent plus importants.**

TF-IDF

Principe : Pénalise la fréquence d'un mot en se basant sur son occurrence dans plusieurs documents:

$$\Rightarrow TF - IDF(w, d) = TF(w, d) \times IDF(w)$$

- **TF (Term Frequency)** : Fréquence d'un mot dans un document donné.

$$TF(w, d) = \frac{\text{nombre d'occurrences de } w \text{ dans } d}{\text{nombre total de mots dans } d}$$

- **IDF (Inverse Document Frequency)** : Mesure la rareté d'un mot dans l'ensemble du corpus

$$IDF(w) = \log\left(\frac{N}{DF(w)}\right)$$

avec N : Nombre de documents ; DF(w) : Nombre de documents contenant w

Latent Semantic Analysis (LSA)

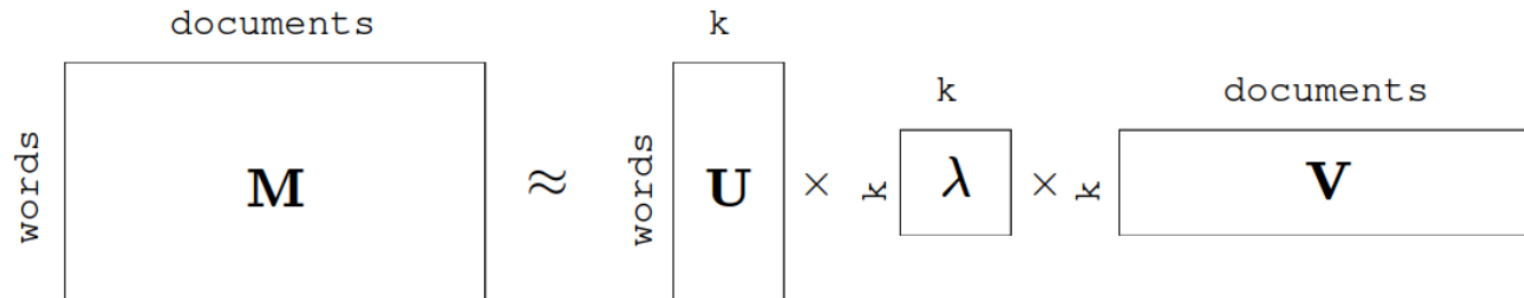
Objectifs :

- Réduction de dimensions de la matrice TF-IDF.
- Capturer les relations sémantiques entre mots et documents.

Idée :

- Extraire des concepts latents à partir des relations entre mots et documents à l'aide d'une **décomposition SVD** :

$$M = U\Lambda V^T$$



LSA : Exemple

Corpus de 6 phrases :

- D1 : Le **chat** dort sur le **canapé**.
- D2 : Le **chien** dort sur le **tapis**.
- D3 : Le **chat** et le chien **jouent** ensemble.
- D4 : La **banque** finance un **projet** immobilier.
- D5 : Le **crédit** est accordé par une **banque**.
- D6 : Le **client** discuté à la **banque**.

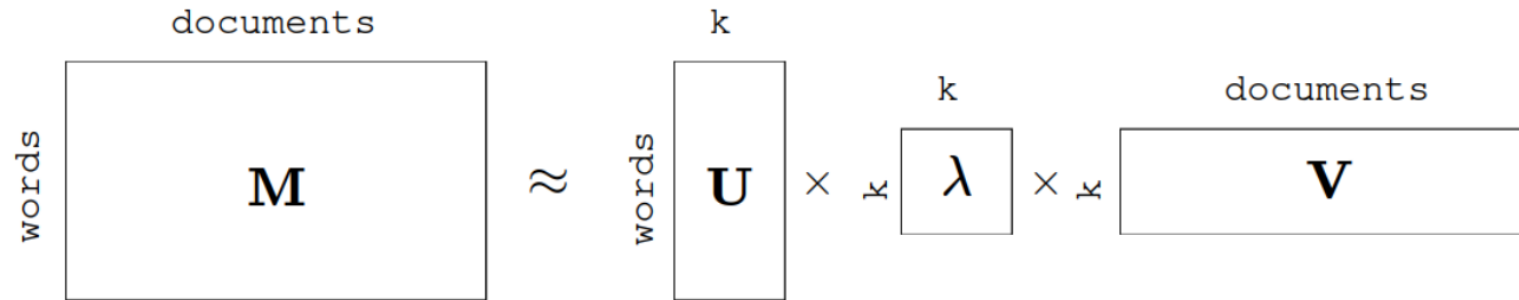
Concepts latents (premiers vecteurs propres) :

- **Concept 1** : *chat, chien, canapé, tapis, jouent ...*
- **Concept 2** : *banque, client, crédit, projet, finance ...*

Interprétation :

- Les documents D1 à D3 sont projetés dans l'espace du **concept animalier**
- Les documents D4 à D6 sont projetés dans l'espace du **concept financier**

Latent Semantic Analysis (LSA)



Matrice U : Chaque mot est représenté par un vecteur de dimension k (k concepts).

Matrice λ : Valeurs singulières : Représentent l'importance de chaque concept dans le corpus.

Matrice V : Chaque document est représenté par un vecteur de dimension k (k concepts).

Capture du sens sémantique : Les mots et documents ayant des sens proches seront projetés sur les mêmes concepts latents (intérêt pour clustering, classification...)

Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

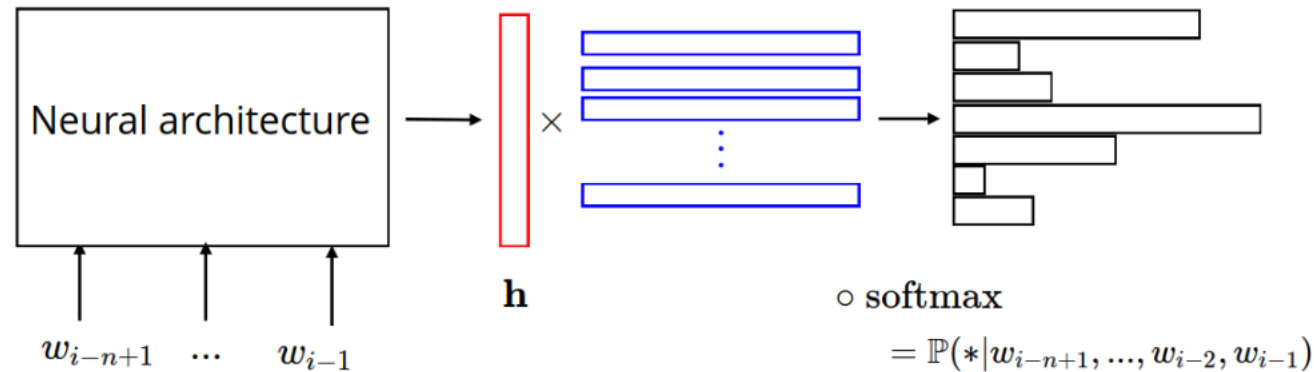
V. L'architecture Transformers

- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

Word2Vec

Word2Vec - Mikolov et al. (2013) :

- **Intuition** : Le sens d'un mot est défini par son contexte $\Rightarrow$ deux mots apparaissant dans les mêmes contextes sont sémantiquement liés.
- **Objectif** : Apprendre des représentations textuelles capturant le sens sémantique et syntaxique des mots en comprenant dans quels contextes apparaissent chaque mot.



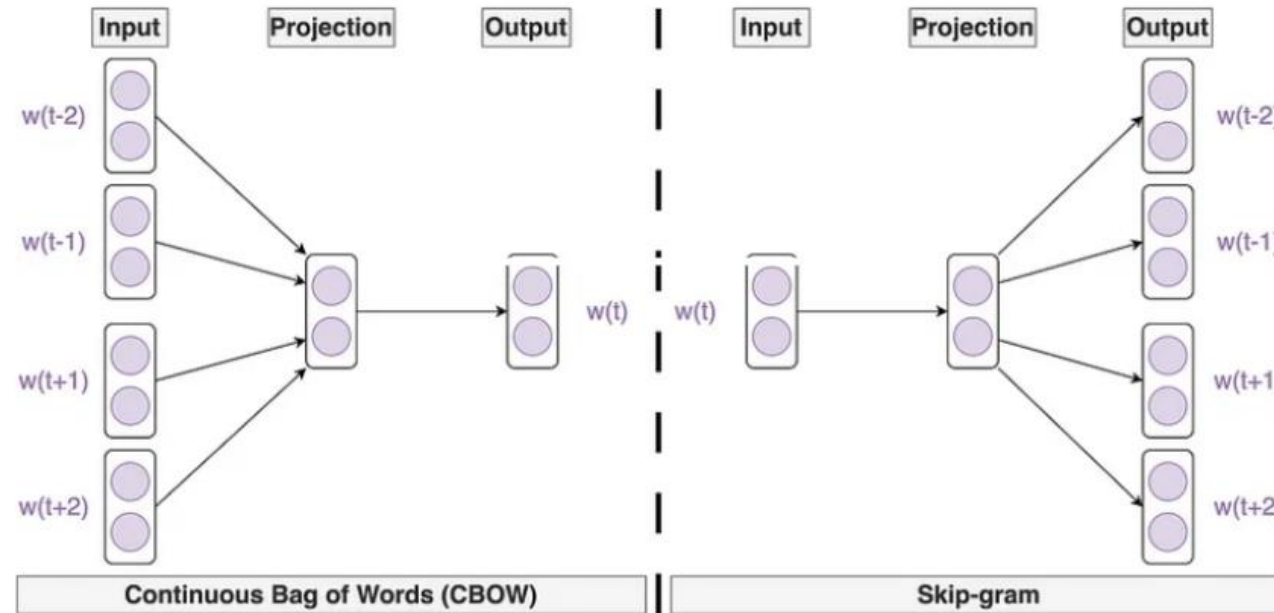
Word2Vec architecture

Word2Vec : Exemple

Dans la phrase « Le chat dort sur le canapé », on veut prédire le mot « chat ».

- L'objectif est de maximiser : $P(w_t = \textit{chat} \mid w_{t-1}, w_{t+1}, w_{t+2}, w_{t+3}, w_{t+4})$
- Durant l'entraînement, le modèle va donc adapter l'embedding de chaque mot pour maximiser cette probabilité conditionnelle.

Word2Vec : Architecture



Continuous Bag of Words (CBOW) :

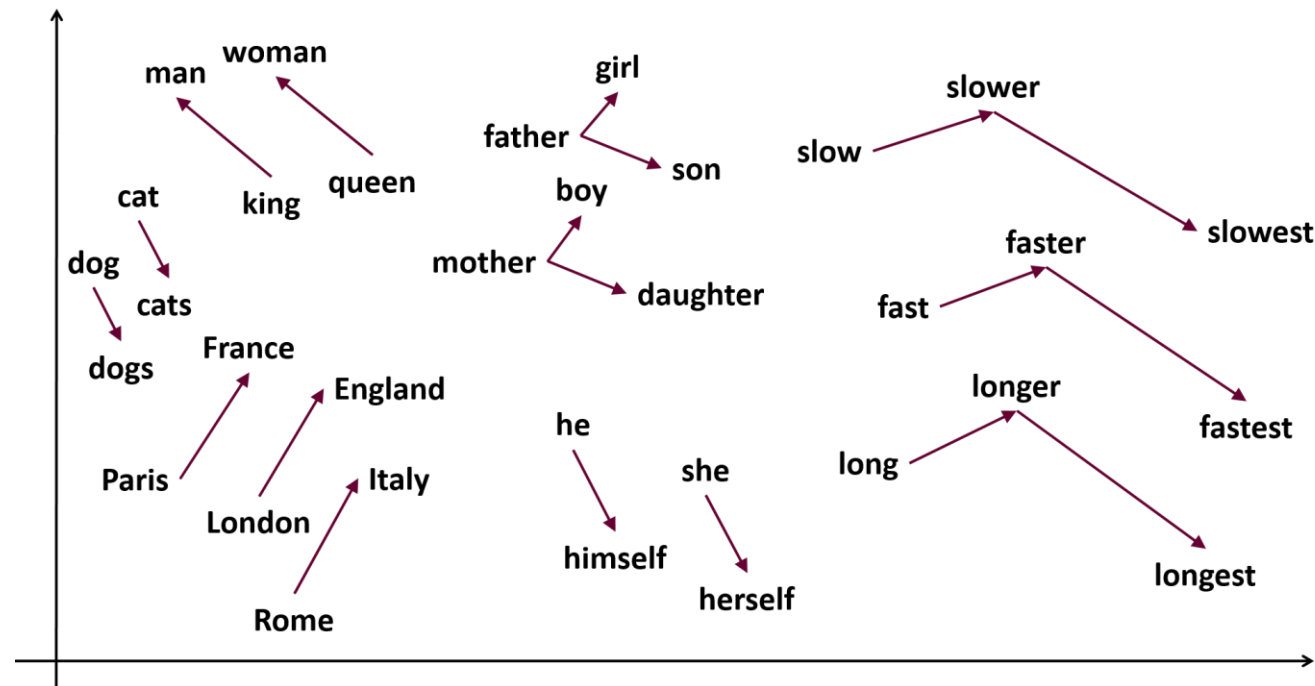
- **Objectif :** Prédire un mot cible w_t à partir de son contexte.

Skipgram :

- **Objectif :** Prédire les mots du contexte $\{w_{t-j}, \dots, w_{t+j}\}$ à partir du mot cible.

Word2Vec : Caractéristiques et avantages

- Capture les **relations sémantiques** et syntaxiques entre les mots.
- Génère des **vecteurs continus et en faibles dimensions**.
- Apprentissage **auto-supervisé** (pas d'annotation nécessaire).
- **Rapide** à entraîner.
- Capacité à résoudre des **analogies** (ex : Roi – homme + femme = Reine).



Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

V. L'architecture Transformers

- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

Importance du contexte

Limite de Word2Vec : Représentation statique de mots.

- Pour l'inférence, **chaque mot est représenté par un embedding**.

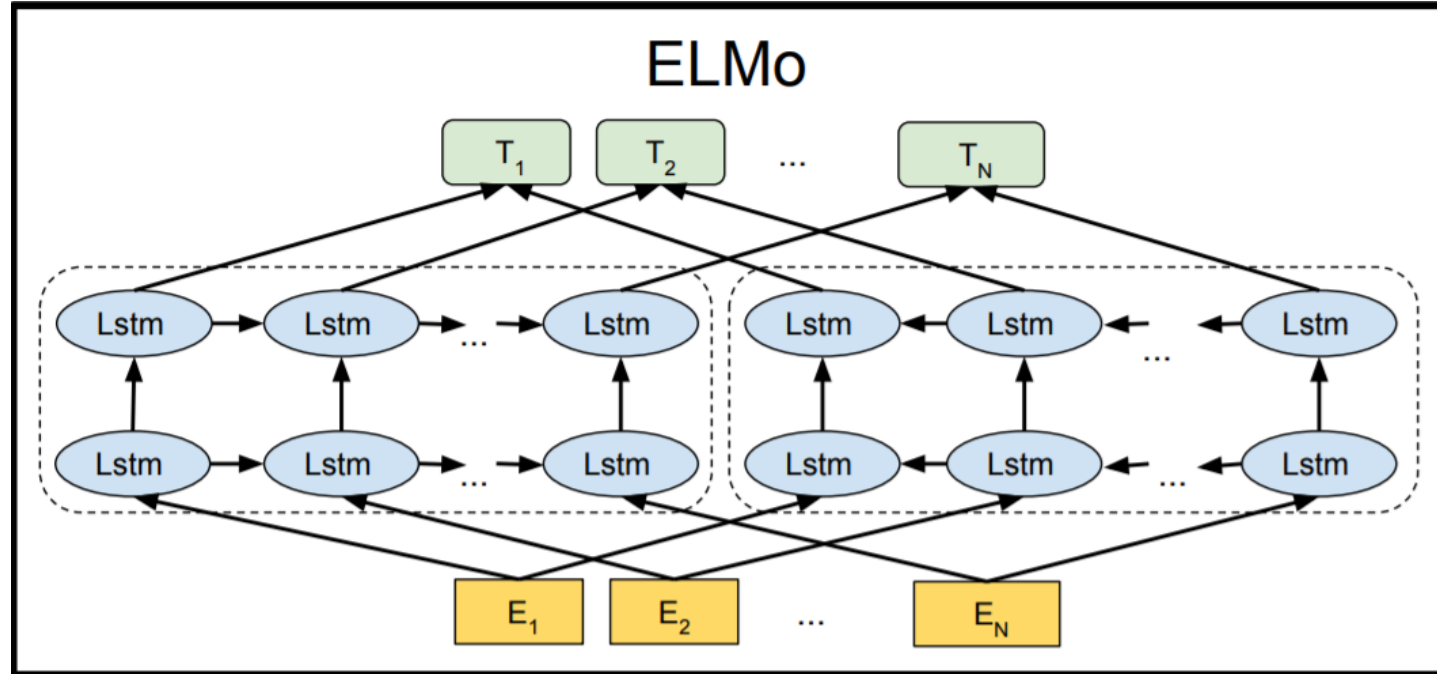
⇒ On **ne process pas le texte** avec un modèle entraîne.

- **Problème** : Un même mot peut avoir des significations différentes en fonction de son contexte
(**Exemple** : « Un banc en bois » - « Un banc de poissons ».)

Embeddings contextuels :

- L'embedding d'un mot dépend de la phrase dans laquelle il apparaît.
- Les embeddings sont appris via des modèles séquentiels (LSTM, Transformer...) pour capturer ce contexte.
- **Pour l'inférence, le modèle est utilisé pour process chaque mot en fonction de son contexte.**

ELMO : Premiers embeddings contextuels



Architecture :

- **BiLSTM** : Deux LSTM qui traitent la séquence dans des directions opposées.
- Deux couches cachées.
- Pour chaque mot, l'embedding final est la concaténation des outputs des deux LSTMs.

ELMO : Premiers embeddings contextuels

Entraînement : Pour optimiser les poids du modèle **ET** les embeddings statiques de mots à fournir en entrée.

- **LSTM forward** : Prédit le mot suivant.
- **LSTM backward** : Prédit le mot précédent.
- **Loss** à minimiser : Cross-Entropy Loss.

Inférence :

- **Embeddings initiaux** : Embeddings statiques de mots.
- Mise à jour des embeddings en fonction des contextes dans les deux sens.
- **Output** : Concaténation des embeddings des deux LSTMs.

Plan du cours

I. Introduction au NLP

II. Premières méthodes de représentation de documents

- Bag of Words (BoW)
- TF-IDF
- LSA

III. Static Word Embeddings

- Word2Vec

IV. Contextual Word Embeddings

- ELMO

V. L'architecture Transformers

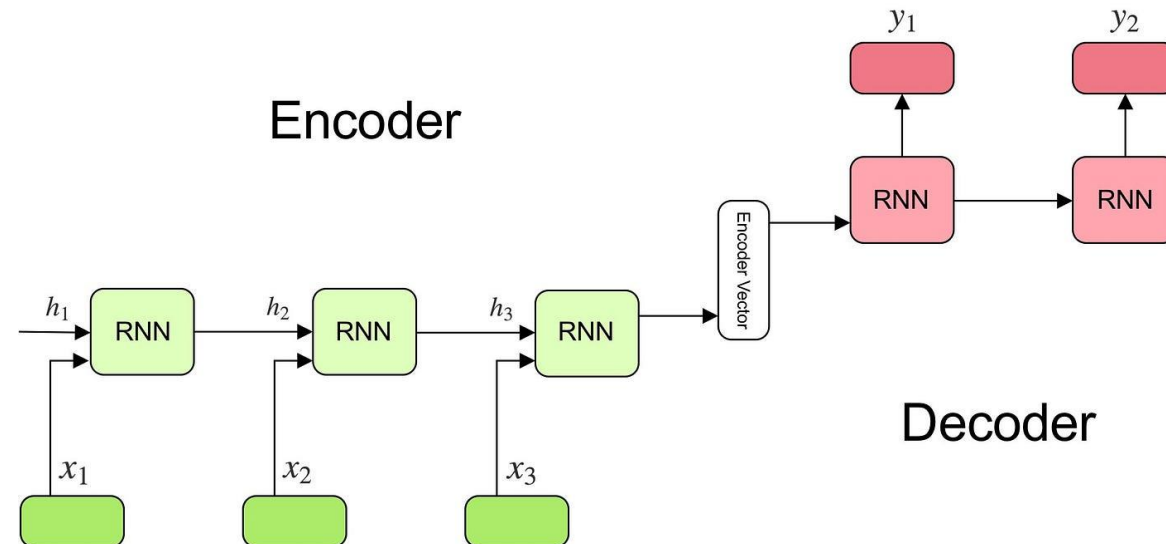
- L'architecture Encoder-Decoder
- Mécanisme d'attention
- Self-attention et introduction des modèles Transformers.
- BERT : Bidirectionnal Encoder Representations from Transformers
- Large Language Models (LLMs)

L'architecture encoder-decoder

Objectif : Traiter les tâches Sequence-to-Sequence (ex : Traduction, résumé, chatbot...)

Architecture (LSTMs ou RNNs) :

- **Encoder :** Traite la séquence d'entrée (séquentiellement).
⇒ Capture les informations dans un vecteur (état caché du LSTM) mis à jour à chaque étape. Le vecteur final est fourni au décodeur.
- **Decoder :** Traite l'information du vecteur fourni par l'encodeur pour créer séquentiellement la phrase attendue en output.



L'architecture encoder-decoder

Soit $(X_1, X_2, \dots, X_T)$ la phrase d'entrée, l'objectif est de générer une séquence $(Y_1, Y_2, \dots, Y_{T'})$ qui maximise la probabilité :

$$\mathbb{P}[(Y_1, Y_2, \dots, Y_{T'}) | (X_1, X_2, \dots, X_T)]$$

Fonctionnement du décodeur :

- L'encodeur fournit un **vecteur de contexte C** (état caché final).
- L'objectif du décodeur est donc de maximiser : $\mathbb{P}[(Y_1, Y_2, \dots, Y_{T'}) | C]$.
- A chaque étape, le décodeur prend en compte le vecteur de contexte ainsi que les éléments générés précédemment et doit donc maximiser :

$$\mathbb{P}[(Y_1, Y_2, \dots, Y_{T'}) | (X_1, X_2, \dots, X_T)] = \prod_{t=1}^{T'} \mathbb{P}[Y_t | (C, Y_1, \dots, Y_{t-1})]$$

Limite : Toute l'information doit être contenue dans un unique vecteur.

⇒ **Mauvaises performances sur des séquences longues.**

Mécanisme d'attention

Objectif : A chaque étape du décodage, spécifier au décodeur les séquences de la phrase d'entrée qui sont pertinentes.

Mécanisme d'attention :

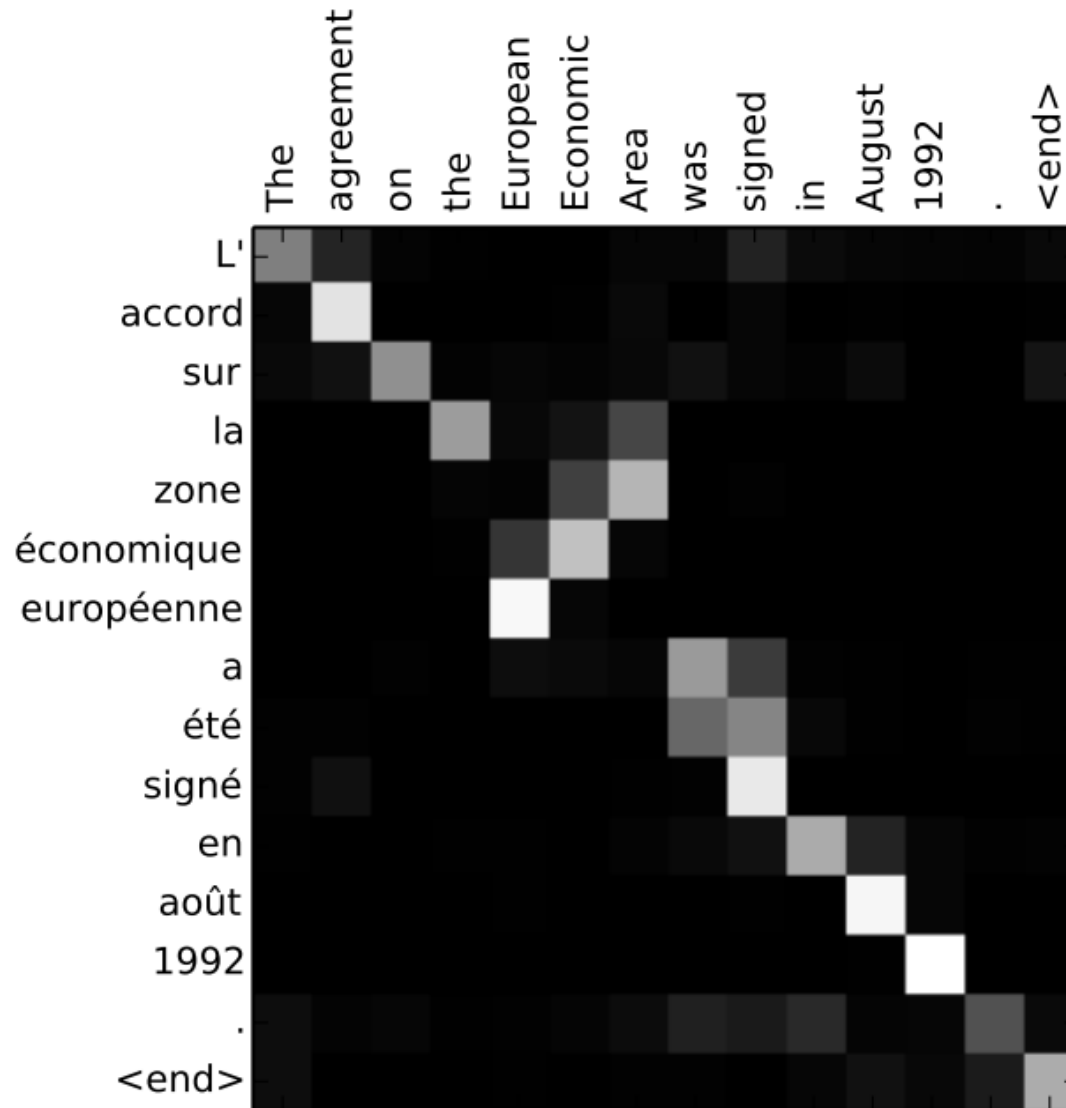
- On accède désormais à tous les états cachés de l'encodeur.
- A chaque étape t , on calcule des vecteurs d'alignement e_{tj} .
⇒ Déterminent si les données d'entrée à la position j matchent avec l'état caché du décodeur autour de la position t .
- Le décodeur effectue un pooling des états cachés pour obtenir l'information pertinente :

$$C_t = \sum_{k=1}^T \alpha_{tk} h_k$$

avec h_j les états cachés et α_{tj} les poids d'attention :

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^T \exp(e_{tk})}$$

Mécanisme d'attention



Attentions calculées pour une tâche de traduction.

Self-attention

Objectif : Calculer l'attention sur les mots d'une même séquence.

⇒ Permet d'attribuer à chaque mot, les informations sur les mots qui lui sont le plus liés au sein de la séquence.

Avantages :

- Permet de capturer les dépendances entre les mots d'une séquence.
- Permet de paralléliser les calculs : Chaque mot peut être traité indépendamment (avantage sur LSTM/RNN).

Self-attention : Fonctionnement

Pour chaque mot, on génère trois vecteurs : Keys (**K**), Queries (**Q**) et Values (**V**) à l'aide de matrices apprises pendant l'entraînement.

Self Attention :

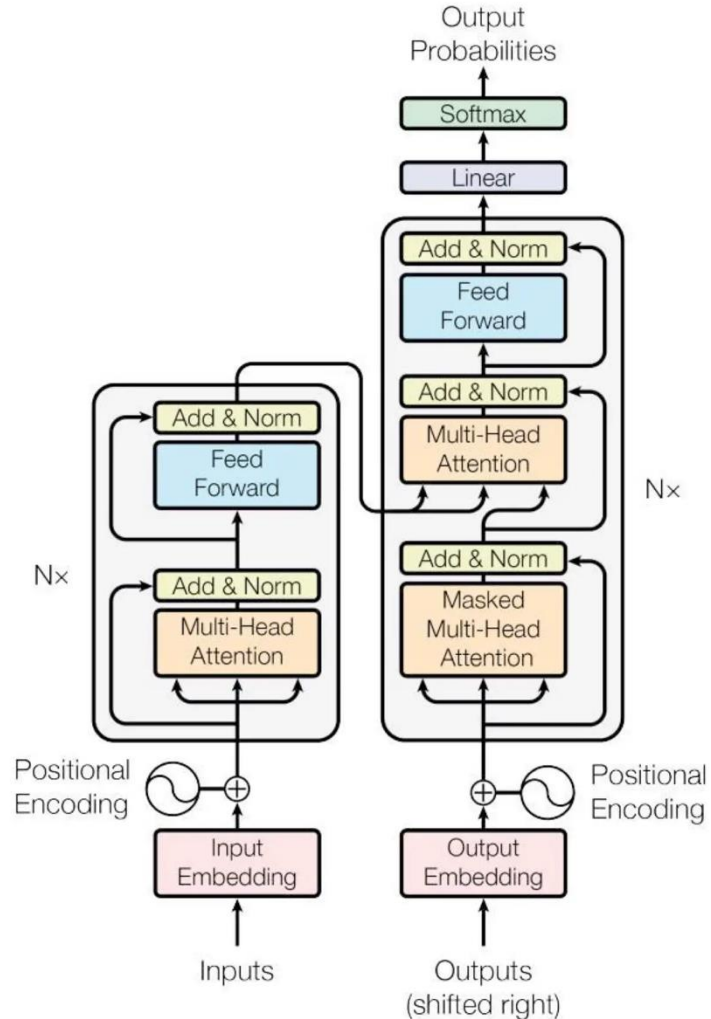
$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d^K}}\right) V$$

- $\frac{QK^T}{\sqrt{d^K}}$: Produit entre query (du mot) et toutes les keys (normalisé).

⇒ Donne une matrice de scores de similarités.

- ***softmax***(.) : Transforme les scores en poids d'attentions.
- ***softmax***(.) · **V** : Attribue à chaque mot une combinaison pondérée des autres mots.

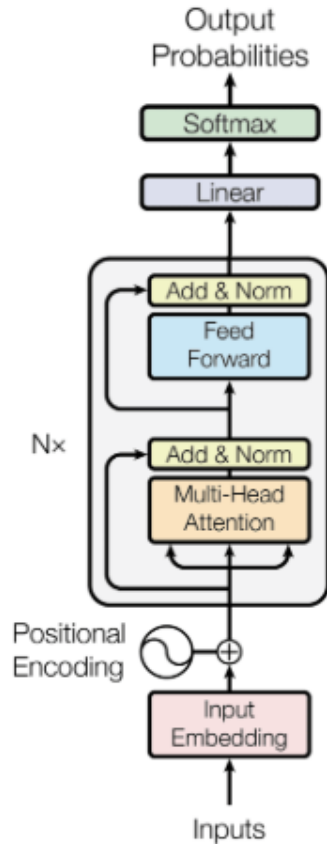
Architecture Transformers



- **Positional Encoding** : Information de positions des tokens.
- **Multi-Head Attention** : Plusieurs têtes d'attention qui capturent des relations différentes.
- **Feedforward Network** : Réseau dense appliqué à chaque token.
- **Etat de l'art actuel** : Architecture des modèles BERT, GPT ...
- **Librairie Transformers** (huggingface) permet d'obtenir les poids des modèles opensource et de les finetuner facilement.

Architecture du modèle Transformers

BERT Model



Architecture du modèle BERT

- **Modèle d'analyse de texte** : Basé uniquement sur l'encodeur du modèle Transformers.
- **Attention bidirectionnelle** : Apprentissage du contexte dans les deux sens.
- **Modèle pré-entraîné** sur des très larges corpus de textes pour une compréhension générale du langage.
- **Etat de l'art** sur de nombreuses tâches grâce aux possibilités de finetuning.

Pré-entraînement et finetuning

Pré-entraînement :

- BERT est pré-entraîné sur des grands corpus de textes (de manière **auto-supervisée**) sur les tâches suivantes :
 - **Masked Language Model (MLM)** : Masque 15% des mots et les prédit.
 - **Next Sentence Prediction** : Prédire si la phrase B suit la phrase A.

⇒ Entraînement très coûteux mais réalisé une seule fois : Les poids sont **disponibles en open-source** (huggingface...)

Finetuning :

- On part des poids pré-entraînés pour les adapter sur une tâche ou sur des données spécifiques :
 - Ajout **d'une couche de sortie** adaptée (ex : Softmax pour la classification).
 - **Finetune sur la tâche spécifique** : nécessite peu de données (quelques milliers).

Tokenization : Réduire la taille du vocabulaire

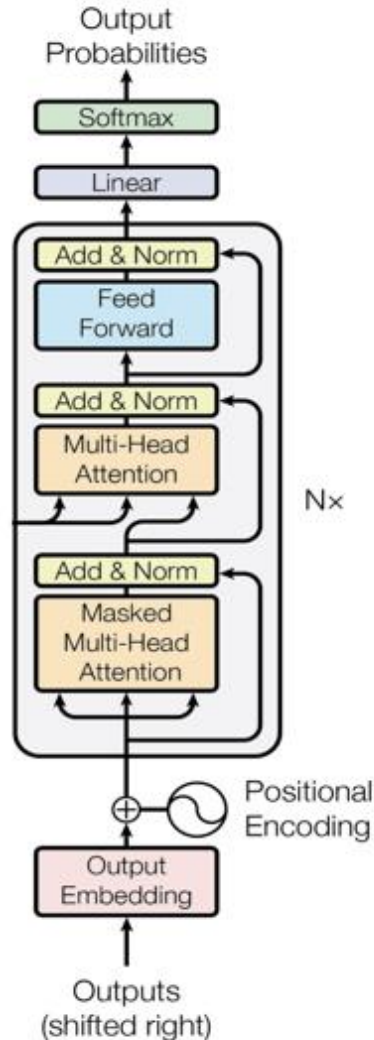
Processus de tokenization : Transformer un texte en unités minimales de sens (**tokens**) qui peuvent être traitées par un modèle de langage.

Tokenization dans BERT :

- Les mots sont séparés en sous unités (subwords).
 - Souvent : Préfixe, base du mot, suffixe.
 - **Exemple** : « *Internationalisation* » → [« *Inter* », « *##national* », « *##isation* ».
- Chaque token est associé à un embedding appris durant l'entraînement.

⇒ La tokenization en subwords est très efficace car elle permet de **réduire fortement la taille du vocabulaire**.

Modèles génératifs



Architecture du modèle GPT

- **Modèle de génération de texte** : Basé uniquement sur le décodeur du modèle Transformers.
- **Unidirectionnel** : Contexte de gauche à droite.
- **Pré-entraînement** : Prédire le prochain token.
- **Modèles LLMs** permettent des performances exceptionnelles (Milliards de paramètre / pré-entraînement conséquent et optimisé).

Pré-entraînement des LLMs (non-supervisé)

Pré-entraînement :

- Tâche de prédiction du prochain token sur de très larges corpus de texte (milliards d'exemples).
- Le modèle apprend la syntaxe, grammaire ainsi que des connaissances.
- Mais incapable de se comporter en assistant (répondre aux questions).
- **Exemple** : Si on lui demande : « Quel est ton prénom? », il répond : « Quel est ton nom de famille.

Entraînement des LLMs (supervisé)

Instruction Fine-tuning :

- On reprend les poids appris pendant la première phase.
- Même tâche de prédiction, mais sur des données très propres sous le format « instruction – réponse ».
- Le modèle a besoin de moins de données car il est déjà pré-entraîné.
- Pendant cette phase, le modèle apprend le comportement un assistant.

Reinforcement Learning from Human Feedback (RLHF) :

- Pour chaque prompt, le modèle génère plusieurs réponses.
- Des annotateurs humains classent ces réponses.
- On entraîne un modèle de récompense $\Rightarrow$ Doit prédire ces classements.
- Le modèle de langage est entraîné à maximiser les récompenses.